

TD 2 : Analyse Lexicale (Théorie des langages et compilation)

Objectifs

Ce TD vise à aider les étudiants à :

- Identifier et comprendre les notions de token, lexème, et catégorie lexicale,
- Manipuler des expressions régulières pour définir des lexèmes,
- S'exercer à la construction manuelle d'automates simples,
- Comprendre le rôle de la table des symboles,
- Identifier des erreurs lexicales.

Exercice 1 : Lexèmes et tokens (rappel de définitions)

Soit le code suivant en pseudo-langage C :

```
if (x123 > 9) { count = count + 1; }
```

1. Listez tous les lexèmes trouvés dans ce code.
2. Donnez pour chaque lexème le **token** correspondant.
3. Identifiez les **catégories lexicales** principales.

Indications :

- Mots-clés : if, else, while, etc.
- Identifiants : noms de variables
- Opérateurs : >, =, +
- Constantes : nombres
- Délimiteurs : (,), {, }, ;

Exercice 2 : Expressions régulières

Pour chaque type de token, donnez une expression régulière qui le reconnaît :

1. Identifiant (lettre ou underscore, suivi de lettres, chiffres ou underscores)
2. Nombre entier positif
3. Opérateur arithmétique : +, -, *, /
4. Mot-clé exact : `while`
5. Chaîne entre guillemets : ex. "bonjour"

Exercice 3 : Lecture pas à pas par un automate (DFA)

On considère un automate fini déterministe pour reconnaître des **entiers positifs** :

- Alphabet : {0-9}
 - États : q0 (initial), q1 (final)
 - Transitions :
 - $q0 \rightarrow q1$ si chiffre non nul (1-9)
 - $q1 \rightarrow q1$ si chiffre (0-9)
1. Décrivez pas à pas comment cet automate traite la chaîne : 3205
 2. Quelle serait l'erreur si on commence par 0 ? (si 0 interdit au début)

Exercice 4 : Table des symboles

Soit le code suivant :

```
int total = 0;
while (total < 10) {
    total = total + 1;
}
```

1. Quels sont les identifiants dans ce code ?
2. Pour chacun, indiquez :
 - Son nom,
 - Son type lexical (ID, mot-clé...),
 - Sa portée (globale, locale),
 - Sa position approximative (ligne ou colonne).
3. Expliquez pourquoi la table des symboles est importante pour les phases suivantes du compilateur.

Exercice 5 : Détection d'erreurs lexicales

Indiquez s'il y a une erreur lexicale dans chacune des lignes suivantes. Si oui, expliquez pourquoi.

1. `int @val = 5;`
2. `string nom = "Jean; (guillemet fermant manquant)`
3. `float 3x = 2.0;`
4. `if (total >= 0)`
5. `#define SIZE 100`